📘 DATABASE CONCEPTS | SQL

SECTION 1: DATABASE CONCEPTS

Term	Definition	Example
Database	A collection of related data stored in an organized manner so that it can be easily accessed, managed, and updated.	A **School Database** containing Student, Teacher, and Marks tables.
DBMS	Software used to create, store, organize, retrieve, and manage data in a database.	**MySQL**, Oracle, Microsoft Access, SQLite.
Table (Relation)	A collection of related data arranged in rows and columns. Each table stores information about one type of entity.	A **STUDENT** table with columns: RollNo, Name, Class, Marks.
Row (Tuple / Record)	A single complete entry in a table. Each row contains all the information about one record.	**101, Rahul, XII-A, 92** is one row in the STUDENT table.
Column (Field / Attribute)	A category of information in a table. Each column stores one type of data.	**Name** and **Marks** are columns in the STUDENT table.
Cardinality	The total number of rows (records) present in a table.	If the STUDENT table has **250 students**, its cardinality is **250**.
Degree	The total number of columns (attributes) present in a table.	If the STUDENT table has **RollNo, Name, Class, Marks**, its degree is **4**.
Domain	The set of valid values that can be stored in a column.	The **Marks** column can store only values from **0 to 100**.
Candidate Key	A column or combination of columns that can uniquely identify each record in a table. A table can have more than one candidate key.	If both **RollNo** and **AdmissionNo** are unique, both are candidate keys.
Primary Key	The candidate key chosen to uniquely identify every record in a table. It cannot contain duplicate or NULL values.	**RollNo** is selected as the Primary Key of the STUDENT table.

Alternate Key	The candidate key(s) that are not selected as the Primary Key.	If **RollNo** is the Primary Key, then **AdmissionNo** becomes the Alternate Key.

SECTION 2: SQL – STRUCTURED QUERY LANGUAGE

What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with Relational Database Management Systems (RDBMS). It is used to create databases, create tables, insert data, retrieve data, update data, and delete data.

Advantages of SQL

- Uses simple English-like commands, making it easy to learn.
- Can store and manage very large amounts of data efficiently.
- It is the standard language used by most Relational Database Management Systems (RDBMS).
- Supports creating databases and tables, retrieving data, inserting records, updating records, and deleting records.
- Provides fast and accurate data retrieval using queries.
- Allows multiple users to access the database securely.
- Portable and supported on almost all major database systems.

Classification of SQL Commands

Type	Full Form	Purpose	Commands
DDL	Data Definition Language	Define/modify structure	CREATE, ALTER, DROP
DML	Data Manipulation Language	Manage data/content	INSERT, UPDATE, DELETE, (SELECT)
DQL	Data Query Language	Retrieve data	SELECT

Introduction to MySQL

MySQL is a popular free and open-source Relational Database Management System (RDBMS) widely used for web applications.

SECTION 3: GENERAL MYSQL COMMANDS

```
SHOW DATABASES;              -- List all databases
USE database_name;           -- Open a database
SHOW TABLES;                 -- List all tables in current DB
DESC table_name;             -- Show structure/description of table
```

SECTION 4: DATA TYPES IN MYSQL

Data Type	Description	Example
INT	Integer numbers	101, -5, 0
FLOAT	Decimal numbers (approximate)	85.5, 99.99
CHAR(size)	Fixed length string (1-255)	CHAR(10) → 'Hello '
VARCHAR(size)	Variable length string	VARCHAR(50) → 'Hello'
DATE	YYYY-MM-DD	'2025-06-15'

SECTION 5: DDL – DATA DEFINITION LANGUAGE

5.1 CREATE DATABASE

Syntax: `CREATE DATABASE database_name;`

```
CREATE DATABASE school;
USE school;   -- To select the database
```

5.2 CREATE TABLE

Syntax:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ...
);
```

Example:

```
CREATE TABLE student (
    roll INT,
    name VARCHAR(50),
    marks FLOAT,
    dob DATE
);
```

5.3 ALTER TABLE

A) ADD – Add new column

Syntax: `ALTER TABLE table_name ADD column_name datatype;`

Example:

```
ALTER TABLE student ADD city VARCHAR(30);
```

→ Add multiple new columns

Syntax: `ALTER TABLE table_name ADD (column_name1 datatype1, column_name2 datatype2);`

Example:

```
ALTER TABLE student ADD (city VARCHAR(30), country CHAR(20));
```

B) DROP – Delete a column

Syntax: `ALTER TABLE table_name DROP COLUMN column_name;`

Example:

```
ALTER TABLE student DROP COLUMN city;
```

→ **Delete multiple columns**

Syntax: `ALTER TABLE table_name DROP COLUMN column_name1, DROP COLUMN column_name2;`

Example:

```
ALTER TABLE student DROP COLUMN city, DROP COLUMN country;
```

C) MODIFY – Change datatype or size

Syntax: `ALTER TABLE table_name MODIFY column_name new_datatype;`

Example:

```
ALTER TABLE student MODIFY name VARCHAR(100);
```

D) CHANGE – Rename column and/or change datatype

Syntax: `ALTER TABLE table_name CHANGE old_name new_name new_datatype;`

Example:

```
ALTER TABLE student CHANGE name student_name VARCHAR(50);
```

5.4 DROP TABLE

Syntax: `DROP TABLE table_name;`

Example:

```
DROP TABLE student;
```

5.5 DROP DATABASE

Syntax: `DROP DATABASE database_name;`

Example:

```
DROP DATABASE school;
```

> **CAUTION:** DROP DATABASE permanently deletes the entire database with all its tables. This operation cannot be undone!

SECTION 6: DML – DATA MANIPULATION LANGUAGE

6.1 INSERT – Add rows

Syntax (all columns): `INSERT INTO table_name VALUES (value1, value2, ...);`
Syntax (specific columns): `INSERT INTO table_name (col1, col2) VALUES (val1, val2);`

```
INSERT INTO student VALUES (101, 'Amit', 85.5, '2005-06-15');
INSERT INTO student (roll, name) VALUES (102, 'Priya');
```

6.2 UPDATE – Modify existing rows

Syntax: `UPDATE table_name SET column1 = value1 WHERE condition;`

```
UPDATE student SET marks = 92 WHERE roll = 101;
```

> **NOTE:** Without WHERE clause, ALL rows will be updated.

6.3 DELETE – Remove rows

Syntax: `DELETE FROM table_name WHERE condition;`

```
DELETE FROM student WHERE roll = 102;
```

> **NOTE:** Without WHERE clause, ALL rows will be deleted.

SECTION 7: DQL – SELECT COMMAND (Data Query)

7.1 Basic SELECT – Fetch/retrieve record(s)

```
SELECT * FROM student;                    -- All columns
SELECT name, marks FROM student;          -- Specific columns
```

7.2 DISTINCT – Remove duplicates

```
SELECT DISTINCT city FROM student;
```

7.3 ALL – Show duplicates (default)

```
SELECT ALL city FROM student;
```

→ WHERE Clause – Operators (to filter records based on conditions)

A) Relational Operators: =, !=, <>, >, <, >=, <=

```
SELECT * FROM student WHERE marks > 80;
SELECT * FROM student WHERE city = 'Delhi';
```

B) Logical Operators: AND, OR, NOT

```
SELECT * FROM student WHERE marks > 75 AND city = 'Delhi';
SELECT * FROM student WHERE city = 'Mumbai' OR city = 'Chennai';
SELECT * FROM student WHERE NOT city = 'Bangalore';
```

C) BETWEEN / NOT BETWEEN

```
SELECT * FROM student WHERE marks BETWEEN 70 AND 90;
SELECT * FROM student WHERE marks NOT BETWEEN 60 AND 80;
```

D) IS NULL / IS NOT NULL

```
SELECT * FROM student WHERE city IS NULL;
SELECT * FROM student WHERE marks IS NOT NULL;
```

E) LIKE – Pattern matching (% = any characters, _ = exactly one character)

```
SELECT * FROM student WHERE name LIKE 'A%';   -- Starts with A
SELECT * FROM student WHERE name LIKE '_a%';  -- Second letter a
```

7.4 ORDER BY – Sorting

Syntax: `SELECT * FROM table ORDER BY column ASC|DESC;` (ASC is default)

```
SELECT * FROM student ORDER BY marks DESC;   -- Highest marks first
```

7.5 GROUP BY – Group rows

```
SELECT city, AVG(marks) FROM student GROUP BY city;
```

7.6 HAVING – Filter groups (used with GROUP BY)

```
SELECT city, COUNT(*) FROM student GROUP BY city HAVING COUNT(*) > 2;
```

> **IMPORTANT NOTES:**
> - WHERE can be used with SELECT, UPDATE, DELETE
> - WHERE cannot be used with GROUP BY
> - WHERE cannot use aggregate functions
> - HAVING is used with GROUP BY instead of WHERE
> - HAVING can use aggregate functions

Mathematical Operators in SQL

Used to perform arithmetic operations on numeric columns.

Operator	Purpose	Example
+	Addition	SELECT marks + 5 FROM student;
-	Subtraction	SELECT marks - 10 FROM student;
*	Multiplication	SELECT marks * 2 FROM student;
/	Division	SELECT marks / 2 FROM student;

Aliasing (AS) – Giving Temporary Names

Used to give a temporary name to a column or table in the result.

```
SELECT name AS StudentName, marks AS Score FROM student;
SELECT AVG(marks) AS AverageMarks FROM student;
```

SECTION 8: AGGREGATE FUNCTIONS

Function	Purpose	Example Query	Output
MIN(column)	Minimum value	SELECT MIN(marks) FROM student WHERE city = 'Delhi';	35 (if 35 is the lowest mark of students in Delhi)
MAX(column)	Maximum value	SELECT MAX(marks) FROM student WHERE city = 'Mumbai';	98 (if 98 is the highest mark of students in Mumbai)
SUM(column)	Sum of values	SELECT SUM(marks) FROM student WHERE class = '12A';	567 (if total marks of all 12A students is 567)
AVG(column)	Average of values	SELECT AVG(marks) FROM student WHERE gender = 'Female';	75.5 (if average marks of female students is 75.5)
COUNT(*)	Count all rows (including NULL)	SELECT COUNT(*) FROM student WHERE city = 'Delhi';	10 (if there are 10 students in Delhi)
COUNT(column)	Count non-NULL values	SELECT COUNT(city) FROM student WHERE marks > 50;	8 (if 8 students have marks greater than 50 and have city value)
COUNT(DISTINCT col)	Count unique non-NULL values	SELECT COUNT(DISTINCT city) FROM student WHERE class = '12A';	3 (if students of 12A class come from 3 different cities)

SECTION 9: MATH FUNCTIONS

Function	Purpose	Example Query	Output

POWER(n, p)	n raised to power p	SELECT POWER(marks, 2) FROM student WHERE roll = 101;	7225 (if marks = 85)
ROUND(n, d)	Round n to d decimals	SELECT ROUND(AVG(marks), 2) FROM student WHERE city = 'Delhi';	75.45 (if average marks of Delhi students is 75.4546, it rounds to 75.45)
MOD(a, b)	Remainder of a/b	SELECT MOD(marks, 10) FROM student WHERE roll = 101;	5 (if marks = 85)

SECTION 10: TEXT FUNCTIONS

Function	Purpose	Example Query	Output
UCASE() / UPPER()	Convert to uppercase	SELECT UCASE(name) FROM student WHERE roll = 101;	'AMIT' (if name = 'Amit')
LCASE() / LOWER()	Convert to lowercase	SELECT LCASE(name) FROM student WHERE roll = 102;	'priya' (if name = 'Priya')
MID() / SUBSTRING() / SUBSTR()	Extract substring	SELECT MID(name, 1, 3) FROM student WHERE roll = 101;	'Ami' (if name = 'Amit')
LENGTH()	Length of string	SELECT LENGTH(name) FROM student WHERE roll = 101;	4 (if name = 'Amit')
LEFT(str, n)	First n characters	SELECT LEFT(name, 2) FROM student WHERE roll = 101;	'Am' (if name = 'Amit')
RIGHT(str, n)	Last n characters	SELECT RIGHT(name, 3) FROM student WHERE roll = 101;	'mit' (if name = 'Amit')
INSTR(str, sub)	Position of substring	SELECT INSTR(name, 'a') FROM student WHERE roll = 102;	4 (if name = 'Priya')
LTRIM()	Remove leading spaces	SELECT LTRIM(city) FROM student WHERE roll = 101;	'Delhi' (if city was ' Delhi')
RTRIM()	Remove trailing spaces	SELECT RTRIM(city) FROM student WHERE roll = 101;	'Delhi' (if city was 'Delhi ')
TRIM()	Remove both sides spaces	SELECT TRIM(city) FROM student WHERE roll = 101;	'Delhi' (if city was ' Delhi ')

SECTION 11: DATE FUNCTIONS

Function	Purpose	Example Query	Output
NOW()	Current date and time	SELECT NOW() FROM student WHERE roll = 101;	'2026-06-18 14:30:00' (current date & time)
DATE()	Extracts date part	SELECT DATE(dob) FROM student WHERE roll = 101;	'2005-06-15' (if dob = '2005-06-15 10:30:00')
MONTH(date)	Month number (1-12)	SELECT MONTH(dob) FROM student WHERE roll = 101;	6 (if dob = '2005-06-15')
MONTHNAME(date)	Month name	SELECT MONTHNAME(dob) FROM student WHERE roll = 101;	'June' (if dob = '2005-06-15')
YEAR(date)	Year	SELECT YEAR(dob) FROM student WHERE roll = 101;	2005 (if dob = '2005-06-15')
DAY(date)	Day of month	SELECT DAY(dob) FROM student WHERE roll = 101;	15 (if dob = '2005-06-15')
DAYNAME(date)	Day name	SELECT DAYNAME(dob) FROM student WHERE roll = 101;	'Wednesday' (if dob = '2005-06-15')

SECTION 12: JOINS (Working with Two Tables)

12.1 EQUI JOIN

Joins two tables using the = operator. Requires explicit condition.

```
SELECT student.name, marks.score
FROM student, marks
WHERE student.roll = marks.roll;
```

12.2 Cartesian Product (CROSS JOIN)

Combines each row of first table with every row of second table. No join condition is used.

```
SELECT * FROM student, marks;
-- OR
SELECT * FROM student CROSS JOIN marks;
```

> **NOTE:** If student has 5 rows and marks has 5 rows, Cartesian product returns 5 × 5 = 25 rows.

SECTION 13: ADDING PRIMARY KEY CONSTRAINT - using CREATE and ALTER

1. Adding Primary Key during CREATE TABLE (at column level)

```
CREATE TABLE student (
    roll INT PRIMARY KEY,
    name VARCHAR(50),
    marks FLOAT,
    dob DATE
);
```

2. Adding Primary Key during CREATE TABLE (at table level – for composite keys)

```
CREATE TABLE marks (
    student_id INT,
    subject_id INT,
    score FLOAT,
    PRIMARY KEY (student_id, subject_id)
);
```

3. Adding Primary Key using ALTER TABLE (after table is created)

```
ALTER TABLE student ADD PRIMARY KEY (roll);
```

IMPORTANT NOTES about Primary Keys:
- PRIMARY KEY automatically implies NOT NULL and UNIQUE
- A table can have only ONE PRIMARY KEY (but it can be a composite key)
- The column(s) chosen as Primary Key must have unique, non-NULL values in every row

---

SECTION 14: QUICK REFERENCE – SUMMARY TABLE

Command	Type	Purpose
CREATE DATABASE	DDL	Create new database
CREATE TABLE	DDL	Create new table
ALTER TABLE	DDL	Change table structure (ADD, DROP, MODIFY, CHANGE)
DROP TABLE	DDL	Delete table
INSERT	DML	Add rows
UPDATE	DML	Modify rows
DELETE	DML	Remove rows
SELECT	DQL	Retrieve data